

Jour 3 : Les algorithmes du Machine Learning et le Fight des IA

Bonjour à toutes et à tous !

On y est. Jour 3, c'est LA journée. Jusqu'ici on a posé le décor (J1) et on a appris à rendre une donnée propre (J2). Aujourd'hui, on remplit l'Arène. On va découvrir tout l'arsenal du Machine Learning classique : les régressions, les arbres, les forêts, le boosting, les SVM, le clustering. Une dizaine d'algos, chacun avec sa personnalité, ses forces, ses pièges.

Et surtout, on apprend à les juger. Parce qu'un algo qui tourne, c'est facile. Un algo dont on sait dire "il est bon, et voilà pourquoi", ça c'est le métier. Donc on commence par les métriques (comment on note un modèle), et tout le reste de la journée on s'en servira.

Cet après-midi, on organise le grand combat : le **Fight des IA**. Même problème, même découpage, même métrique, et on fait s'affronter tous les algos sur un leaderboard (un tableau de classement). Celui qui gagne, on saura dire pourquoi il gagne. C'est parti.

Coupez-moi la parole, aucune question n'est bête, surtout aujourd'hui où on enchaîne beaucoup de notions.

1 - Révisions rapides

Hier, on a passé la journée sur la donnée. Le réflexe à garder en tête :

- On a appris à repérer et corriger les **valeurs manquantes** (les fameux `NaN`) et les **valeurs aberrantes** (un âge de 200 ans, un prix négatif).
- On a vu la **multicolinéarité** : deux colonnes qui disent la même chose (surface en m2 et surface en pieds carrés), ça perturbe certains modèles.
- On a terminé sur la **PCA** (Analyse en Composantes Principales) : compresser beaucoup de colonnes en quelques-unes qui gardent l'essentiel de l'information.

Rappel express : la PCA ? Une technique qui projette des données à beaucoup de dimensions (500 colonnes) sur un petit nombre d'axes (2 ou 3) qui captent le maximum de variance. On l'a utilisée pour visualiser et pour réduire le bruit.

Le pont vers aujourd'hui : maintenant qu'on sait sortir une donnée propre et bien comprise, on peut **enfin entraîner des algos dessus**. Une donnée sale, le meilleur algo du monde la transforme en bouillie. Une donnée propre, et là on peut comparer les algos sur un pied d'égalité. C'est exactement ce qu'on fait aujourd'hui.

Planning de la Journée

1. **La méthode + les métriques** : les étapes d'un modèle prédictif, et comment on note un modèle (matrice de confusion, precision/recall, ROC, lift, R2/MAE/RMSE)
2. **Régression linéaire** : simple, multiple, et là où le linéaire craque

3. **Régression polynomiale + régularisation** : LASSO, Ridge, et un mot sur les séries temporelles
4. **Régression logistique** : le scoring et le seuil de décision
5. **Clustering** : KMeans et classification hiérarchique (notre seul bloc non-supervisé)
6. **Arbres de décision + Naive Bayes** : lisible, et redoutable sur le texte
7. **Random Forest + Gradient Boosting** : le bagging contre le boosting
8. **SVM** : la marge maximale et le kernel trick, puis la règle "quel algo essayer en premier"
9. **L'après-midi (À votre tour)** : 4 problèmes réels, puis le grand Fight des IA

Chaque notion du matin suit le même rythme : je pose le concept, je montre le code complet, et on observe ensemble ce qui se passe quand on bouge un paramètre. Puis on enchaîne. On va vite, mais on ne laisse personne sur le bord de la route.

2 - La méthode et les métriques : comment on note un modèle

Imaginez que vous codez un détecteur de cancer. Vous le lancez, [scikit-learn](#) vous affiche fièrement **99% d'accuracy** ! Vous êtes content ? Bah vous ne devriez pas 😞. Si dans vos données 99% des patients sont sains, un modèle qui répond toujours "sain" sans rien apprendre fait aussi 99%. Et il rate 100% des vrais malades. Catastrophe.

Donc avant de découvrir le moindre algo, apprenons à les juger. Sinon on avance à l'aveugle.

L'essentiel des étapes et des métriques

D'abord, le squelette d'un projet prédictif. On l'a déjà vu en J1, mais on se rafraîchit la mémoire car c'est la colonne vertébrale de toute la journée :

1. **Cadrer le problème** : qu'est-ce qu'on prédit ? Une catégorie (classification) ou un nombre (régression) ?
2. **Préparer la donnée** : nettoyer, mettre à l'échelle (tout le contenu du J2).
3. **Séparer train / test** : on entraîne sur une partie, on juge sur une partie jamais vue.
4. **Entraîner** : `.fit()`.
5. **Prédire** : `.predict()`.
6. **Évaluer** : avec la BONNE métrique. C'est tout l'enjeu de cette section.

Et ce qu'on évalue se sépare en deux mondes, selon ce qu'on va prédire.

Monde 1 : la classification (on prédit une catégorie).

D'abord, à quoi ça sert dans la vraie vie ? La classification, c'est partout dès qu'on doit ranger quelque chose dans une boîte :

- ce mail est-il un **spam** ou pas (Gmail le fait des milliards de fois par jour),
- cette transaction est-elle une **fraude** ou normale (votre banque, en temps réel, pendant que vous payez),
- cette tumeur est-elle **maligne** ou bénigne (aide au diagnostic médical),

- ce client va-t-il **résilier** son abonnement le mois prochain ou rester (le fameux *churn* en marketing).

À chaque fois, la sortie est une étiquette, pas un nombre. Et pour noter ce genre de modèle, le point de départ, c'est la **matrice de confusion** : un tableau qui croise ce que le modèle a prédit avec la vérité.

C'est quoi une matrice de confusion ?

C'est un tableau 2x2 (pour 2 classes) qui compte 4 choses :

- VP - les vrais positifs : prédit "positif" et c'était effectivement "positif",
- VN - les vrais négatifs : prédit "négatif" et c'était effectivement "négatif",
- FP - les faux positifs : prédit "positif" à tort, une fausse alerte,
- FN - les faux négatifs : prédit "négatif" à tort, on a raté un vrai cas.



Confusion Matrix

Predicted Class

		Predicted Class	
		Positive	Negative
Actual Class	Positive	True Positive	False Negative
	Negative	False Positive	True Negative

À partir de ces 4 chiffres, on fait tous nos calculs :

Instant Notions : les métriques de classification

- **Accuracy** : proportion de bonnes réponses, $(VP + VN) / total$. Elle est simple, mais trompeuse si les classes sont déséquilibrées.
- **Precision** : parmi ce que j'ai prédit positif, combien étaient vraiment positifs ? $VP / (VP + FP)$. Répond à "quand je crie au loup, ai-je raison ?".
- **Recall (rappel)** : parmi tous les vrais positifs, combien j'en ai attrapés ? $VP / (VP + FN)$. Répond à "est-ce que je rate des vrais cas ?".
- **F1-score** : la moyenne harmonique de précision et recall, $2 * (P * R) / (P + R)$. Un seul chiffre entre 0 et 1 qui équilibre les deux, utile quand les classes sont déséquilibrées. (Moyenne harmonique : une moyenne qui tire vers le bas dès qu'une des deux valeurs est faible, donc impossible d'avoir un bon F1 avec une précision ou un recall pourri.)

Le couple precision/recall, c'est un arbitrage. Un détecteur de cancer veut un **recall élevé** (ne rater aucun malade, quitte à avoir quelques fausses alertes). Un filtre anti-spam veut une **precision élevée** (ne jamais mettre un vrai mail important en spam, quitte à laisser passer quelques spams). On choisit selon le coût de l'erreur.

Mais du coup pourquoi pas toujours viser les deux à fond ?

=> Parce qu'ils tirent dans des sens opposés. Si tu prédis "positif" plus souvent pour ne rien rater (recall up), tu te trompes plus souvent (precision down). C'est un curseur, pas deux boutons indépendants.

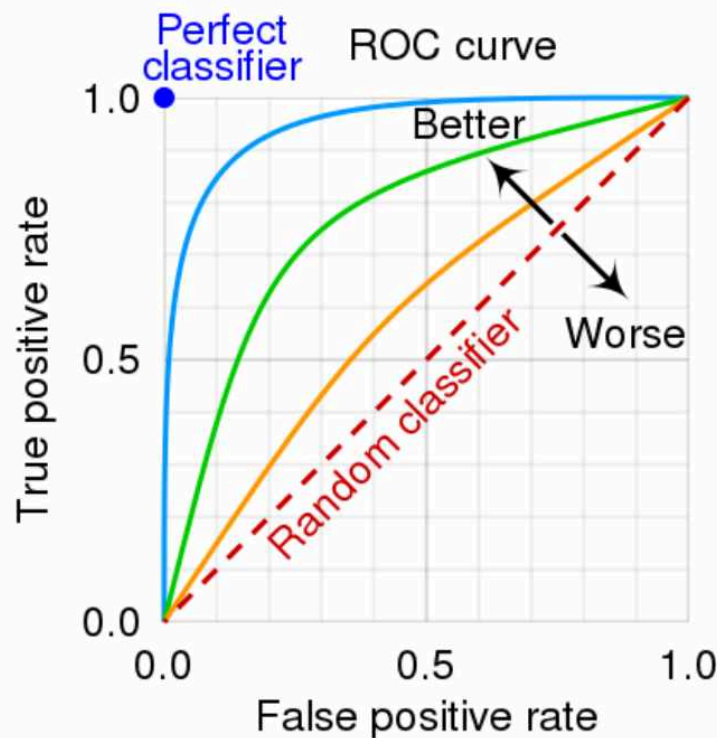
ROC et AUC. Quand un modèle sort une probabilité (genre "70% de chances que ce soit un spam"), on choisit un seuil pour décider. La **courbe ROC** trace, pour tous les seuils possibles, le taux de vrais positifs contre le taux de faux positifs. L'**AUC** (Area Under the Curve) résume cette courbe en un seul nombre.

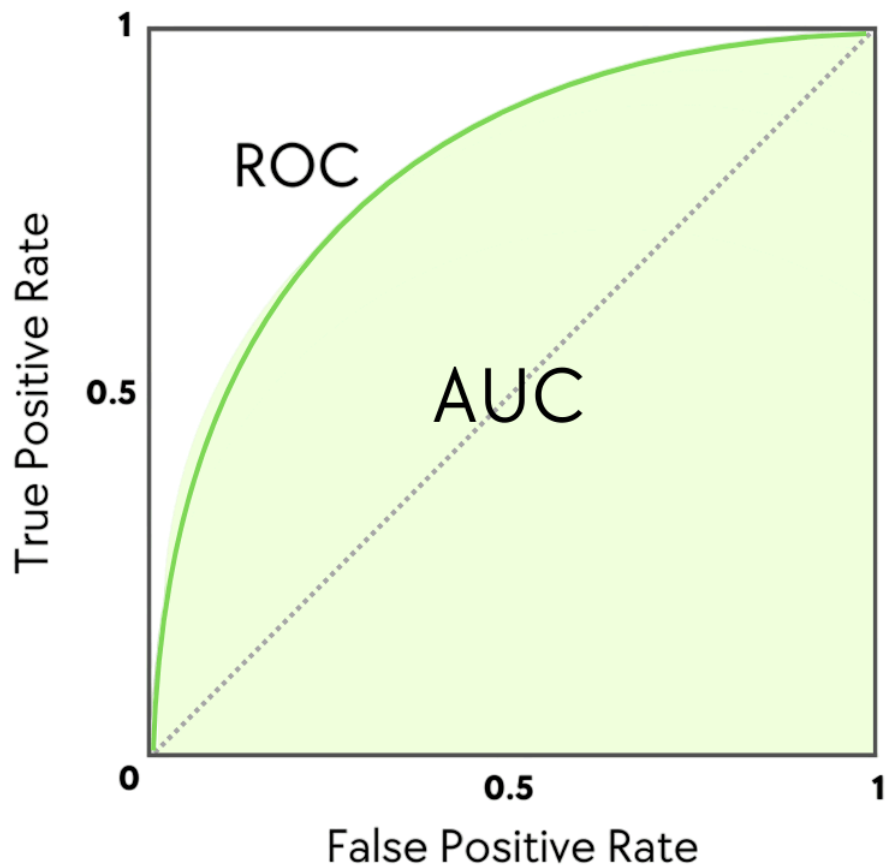
AUC, c'est quoi exactement ? L'aire sous la courbe ROC, entre 0 et 1.

AUC = 0.5, c'est le hasard pur (à pile ou face).

AUC = 1, c'est le modèle parfait.

En pratique, au-dessus de 0.8 c'est bon, au-dessus de 0.9 c'est très bon.

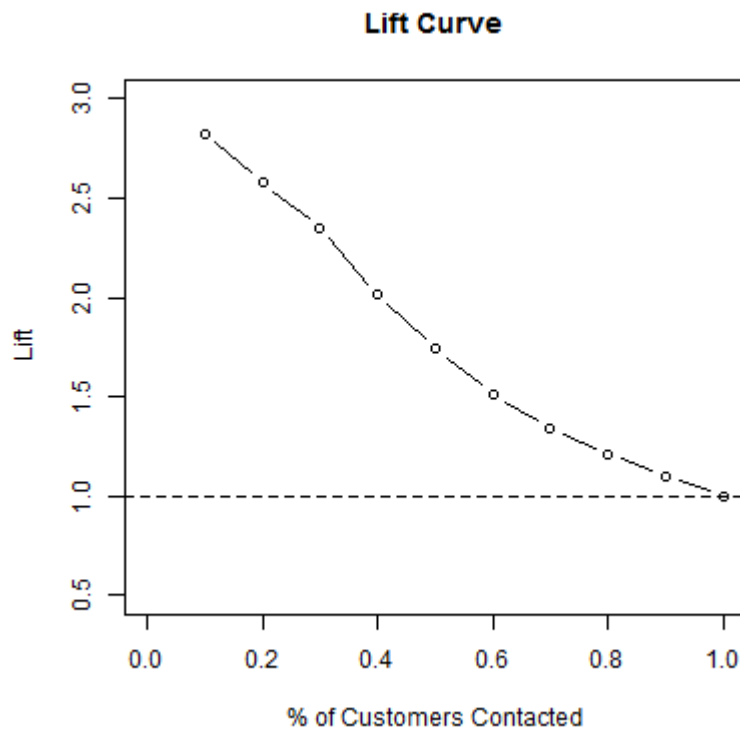




La courbe de lift.

Très utilisée en marketing et en scoring. Elle répond à : "si je contacte les 10% de clients que mon modèle juge les plus prometteurs, combien de fois mieux je fais que si je les contactais au hasard ?". Un lift de 3 sur le premier décile (les 10% les mieux notés par le modèle), ça veut dire 3 fois plus de conversions que l'aléatoire. C'est l'argument qui parle aux décideurs business.

Lift, en anglais : soulever, relever.



Monde 2 : la régression (on prédit un nombre).

Pareil, ancrons-le dans le concret. La régression, c'est dès qu'on prédit une **quantité** sur une échelle continue :

- le **prix** de vente d'un appartement à partir de sa surface et son quartier (ce que fait un site type SeLoger pour estimer un bien),
- le **chiffre d'affaires** de demain pour ajuster les stocks,
- la **consommation électrique** d'une ville à 18h pour dimensionner le réseau,
- le **temps de livraison** estimé qu'on vous affiche quand vous commandez en ligne.

La sortie est un nombre, pas une catégorie. Du coup, plus de matrice de confusion : on mesure l'écart entre la valeur prédite et la vraie valeur.

Instant Notions : les métriques de régression

- **MAE** (Mean Absolute Error) : l'erreur moyenne en valeur absolue. "Je me trompe en moyenne de 25 000 euros sur le prix." Lisible, dans l'unité du problème.
- **RMSE** (Root Mean Squared Error) : la racine de la moyenne des erreurs au carré. Comme la MAE, mais qui **pénalise fort les grosses erreurs** (à cause du carré). Une erreur de 100k compte beaucoup plus que quatre erreurs de 25k.
- **R2 (coefficient de détermination)** : entre 0 et 1 (parfois négatif si le modèle est nul). Proportion de la variance (à quel point les valeurs s'étalent autour de leur moyenne) expliquée par le modèle. $R^2 = 0.85$ veut dire "mon modèle explique 85% des variations du prix". Proche de 1 = excellent, proche de 0 = inutile.

Le choix MAE vs RMSE dépend de votre tolérance aux grosses erreurs. Pour un prix immobilier, une grosse erreur ponctuelle est très coûteuse : la RMSE est plus parlante. Pour un délai de livraison moyen, la MAE suffit.

Assez de théorie, lisons une vraie matrice de confusion. Le code va monter un cas volontairement caricatural pour démasquer l'accuracy :

- inventer 1000 transactions dont seulement 20 fraudes (un cas très déséquilibré, comme dans la vraie vie),
- créer un modèle "paresseux" qui prédit bêtement "tout normal", sans rien apprendre,
- afficher sa matrice de confusion, puis le rapport complet avec precision et recall par classe.

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np

# Vérité terrain : 980 transactions normales (0), 20 fraudes (1)
y_true = np.array([0]*980 + [1]*20)

# Notre modèle "paresseux" : il prédit TOUT normal (jamais de fraude)
y_pred_paresseux = np.array([0]*1000)

print("Matrice de confusion (lignes = vérité, colonnes = prédiction) :")
print(confusion_matrix(y_true, y_pred_paresseux))

# zero_division=0 : évite un warning quand une classe n'est jamais prédite
print(classification_report(y_true, y_pred_paresseux, zero_division=0))
```

Devrait afficher quelque chose comme :

```
Matrice de confusion (lignes = vérité, colonnes = prédiction) :
[[980   0]
 [ 20   0]]

              precision    recall  f1-score   support

     0       0.98         1.00         0.99         980
     1       0.00         0.00         0.00          20

 accuracy                   0.98         1000
```

Regardez bien. **98% d'accuracy**, et pourtant le modèle n'a détecté **aucune fraude** (recall de 0 sur la classe 1). L'accuracy ment, parce que les classes sont déséquilibrées. La precision et le recall, eux, crachent la vérité tout de suite. C'est exactement le piège du "99% d'accuracy" du début.

On bouge un paramètre : remplaçons le modèle paresseux par un modèle qui attrape 15 fraudes sur 20 mais lève 10 fausses alertes :

```
# 965 vrais négatifs, 15 fraudes attrapées, 5 ratées, 10 fausses alertes
y_pred_correct = np.array([0]*965 + [1]*10 + [0]*5 + [1]*15)
y_true = np.array([0]*975 + [1]*25)
```

En relançant le `classification_report`, on va remarquer un truc intéressant. La classe 1 passe d'un recall de 0 (le modèle paresseux ne voyait aucune fraude) à un recall de 0.60 : il en attrape maintenant 15 sur 25. Et c'est super parlant, parce que ce second modèle a une accuracy brute plus basse que le paresseux, mais pour une banque il vaut infiniment mieux : il rapporte vraiment des fraudes au lieu de toutes les laisser filer. L'accuracy seule n'aurait jamais montré cette différence.

Pour aller plus loin

Un piège récurrent en entretien : "accuracy haute = bon modèle ?" Déroulons le raisonnement étape par étape, parce que c'est exactement ce qu'on attend de vous.

D'abord, la réponse de base, c'est ce qu'on vient de voir : non, sur des classes déséquilibrées l'accuracy ment, on regarde precision et recall.

Mais il y a un cran au-dessus. Sur des données vraiment très déséquilibrées (fraude, maladie rare, défaut industriel), on ne regarde même plus l'accuracy du tout.

À la place, on regarde la **precision-recall curve** plutôt que la ROC.

La precision-recall curve ? La courbe qui trace la precision en face du recall pour tous les seuils possibles, sans jamais regarder les vrais négatifs. On la préfère à la ROC ici parce que la ROC peut rester flatteuse quand les négatifs écrasent tout : avec 99% de cas négatifs, le taux de faux positifs reste minuscule même quand le modèle est mauvais sur la classe rare.

Et on calcule souvent le **F-beta** : une version du F1 qui pondère plus le recall ou plus la precision selon le métier (recall prioritaire pour un cancer, precision prioritaire pour un anti-spam).

La leçon de fond, et c'est ça qu'on retient : "quelle métrique je regarde" est une décision métier, pas un détail technique. On y reviendra en J4 sur l'arbitrage du combat.

Pour creuser : [StatQuest - ROC and AUC](#) (~14 min) : le meilleur point de départ pour comprendre visuellement ce que mesure la courbe ROC, et pourquoi l'AUC résume mieux qu'un seuil fixe.

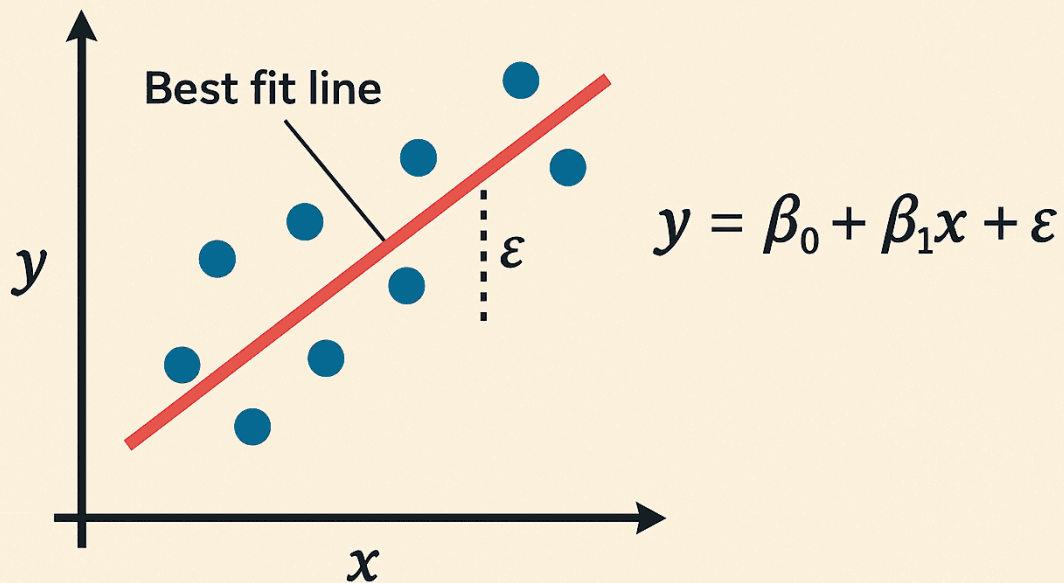
On sait maintenant juger un modèle, c'est notre boussole pour toute la journée. Premier algo : le plus simple, le plus ancien, et le plus utilisé au monde. La régression linéaire. On commence par là parce que tout le reste se compare à elle.

3 - Régression linéaire : simple et multiple

Scénario : une agence immobilière veut estimer le prix d'un appartement. Elle a un historique : surface, nombre de pièces, étage, et le prix de vente final.

La question naturelle : *peut-on tracer une droite qui relie la surface au prix ?* Plus c'est grand, plus c'est cher, ça semble linéaire. C'est exactement l'intuition de la régression linéaire.

LINEAR REGRESSION



L'essentiel

La **régression linéaire** cherche la droite (ou le plan, en plusieurs dimensions) qui colle le mieux aux données. En une variable, c'est `prix = a * surface + b`. Le modèle apprend `a` et `b` pour minimiser l'écart total aux points.

Deux mots à bien visualiser sur cette droite :

La pente (`a`) et l'ordonnée à l'origine (`b`), concrètement ?

- La **pente** `a`, c'est l'inclinaison de la droite : de combien le prix monte quand la surface augmente de 1 m². Une pente de 3000, ça veut dire "+1 m² = +3000 euros".
Une pente forte c'est une droite qui grimpe vite,
Une pente nulle c'est une droite plate (la surface n'influence rien).
- L'**ordonnée à l'origine** `b`, c'est la hauteur où la droite croise l'axe vertical, là où la surface vaut 0.
C'est le "prix de départ" du modèle, son point d'ancrage avant même d'ajouter le moindre m².

Visuellement : `b` dit où la droite démarre, `a` dit à quel point elle monte.

Régression "simple" vs "multiple" ? Simple = une seule variable d'entrée (juste la surface). Multiple = plusieurs variables (surface, pièces, étage, quartier...). Le principe reste identique, seul le nombre de dimensions change.

Et ce passage en "plusieurs dimensions" mérite une image concrète. Avec une seule variable (la surface), on ajuste une **droite** dans un plan : un axe pour la surface, un axe pour le prix.

Ajoutons une deuxième variable, le nombre de pièces. Maintenant on a trois axes : surface, pièces, prix. La relation n'est plus une droite mais un **plan incliné** qui flotte dans cet espace 3D, comme une feuille de papier rigide posée en biais au-dessus du sol. Chaque appartement est un point, et le modèle cherche le plan qui passe au plus près de tous ces points.

Avec 10 variables (comme le dataset ci-dessous), on ne peut plus le dessiner, mais c'est exactement la même idée : un "plan" dans un espace à 11 dimensions. On ne le visualise pas, on fait confiance aux maths, et ça marche pareil.

Voici une régression linéaire multiple complète sur le dataset diabetes livré avec scikit-learn (prédire la progression d'une maladie à partir de 10 mesures médicales) :

random_state=42, c'est quoi ? Ça fige le tirage aléatoire pour que tout le monde obtienne le même découpage train/test : indispensable pour comparer des résultats reproductibles. La valeur 42 est une convention, n'importe quel entier ferait l'affaire.

Voici ce qu'on va enchaîner dans le code :

- charger le dataset diabetes (442 patients, 10 mesures médicales, cible = progression de la maladie),
- séparer en train (80%) et test (20%),
- entraîner la régression linéaire sur le train (`.fit()`),
- prédire sur le test (`.predict()`),
- juger avec les 3 métriques de régression de la section 2 (R2, MAE, RMSE),
- et lire les coefficients appris, le superpouvoir du linéaire.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score

# 442 patients, 10 mesures (âge, IMC, pression...), cible = progression de la maladie
X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

modele = LinearRegression()
modele.fit(X_train, y_train)
y_pred = modele.predict(X_test)

# On juge avec les 3 métriques de régression vues en section 2
print(f"R2 : {r2_score(y_test, y_pred):.3f}")
print(f"MAE : {mean_absolute_error(y_test, y_pred):.1f}")
print(f"RMSE : {root_mean_squared_error(y_test, y_pred):.1f}")

# Le gros avantage du linéaire : on peut LIRE les coefficients
print("Coefficients (poids de chaque variable) :", modele.coef_.round(1))
```

Devrait afficher quelque chose comme :

```
R2      : 0.452
MAE     : 42.8
RMSE    : 53.9
```

Un R2 de 0.45, ce n'est ni nul ni génial : le modèle capte un peu moins de la moitié des variations. C'est honnête pour un modèle aussi simple, et ça nous donne une **baseline** (une référence de base à battre).

C'est quoi une baseline ? Le score de référence le plus simple possible. Tout modèle plus compliqué doit faire mieux que la baseline, sinon il ne sert à rien. En régression, la régression linéaire est la baseline reine.

Le superpouvoir du linéaire : l'interprétabilité. Chaque coefficient dit "quand cette variable augmente de 1, le prix bouge de tant". Un banquier, un médecin, un juge peuvent comprendre et auditer la décision. Aucun réseau de neurones n'offre ça aussi clairement. C'est pour ça que dans les secteurs régulés (banque, santé), le linéaire reste roi.

On touche le terrain : ajoutons ces deux lignes après le `fit` pour voir le poids de chaque variable nommée :

```
noms = load_diabetes().feature_names
for nom, poids in zip(noms, modele.coef_):
    print(f"{nom:>5} : {poids:+.1f}")
```

On va remarquer que `bmi` (l'indice de masse corporelle) et `s5` ressortent avec les plus gros poids : ce sont les variables qui pèsent le plus dans la prédiction de la maladie. Et c'est là que l'interprétabilité devient passionnante : une variable avec un poids **négatif** tire la prédiction vers le bas quand elle augmente. Un médecin lit ça directement comme "ce facteur protège" ou "ce facteur aggrave". Aucun réseau de neurones ne donne une lecture aussi nette.

Creusons

La régression linéaire fait des **hypothèses fortes**, et c'est là qu'elle craque. Elle suppose que la relation est une droite. Or beaucoup de phénomènes ne le sont pas : le prix d'une maison n'augmente pas indéfiniment avec la surface (un château de 2000 m2 ne vaut pas 40 fois un studio de 50 m2). La consommation électrique en fonction de la température fait un U (on chauffe quand il fait froid, on climatise quand il fait chaud). Sur ces cas, une droite passe au milieu et se trompe partout.

Deuxième limite : elle est **sensible aux valeurs aberrantes**. Un seul point très loin tire toute la droite vers lui (à cause de l'erreur au carré qu'elle minimise). D'où l'importance du nettoyage de J2.

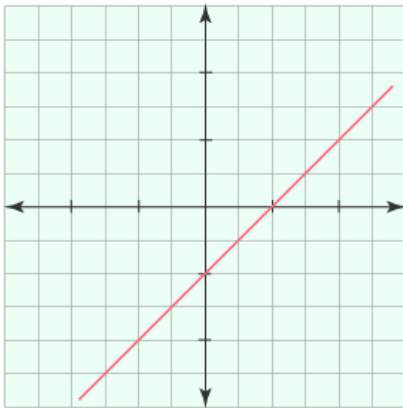
Comment savoir si le linéaire suffit ? On trace les **résidus** (les erreurs, prédit moins réel). S'ils sont répartis au hasard autour de zéro, le linéaire est adapté. S'ils forment une courbe (un U, une vague), c'est le signal qu'il manque de la non-linéarité, et c'est exactement le sujet de la section suivante.

On a notre baseline et on sait quand elle craque (relations non linéaires). La suite logique : comment courber la droite pour épouser des relations plus complexes, sans tomber dans le piège inverse.

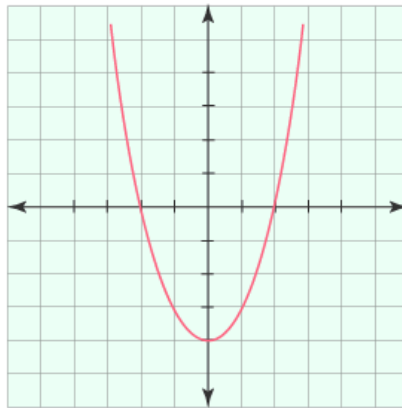
4 - Régression polynomiale et régularisation

Reprenons la consommation électrique en U. Une droite est nulle dessus. Mais si on autorisait le modèle à tracer une **courbe** ? Une parabole épouserait le U sans problème. C'est l'idée de la régression polynomiale : on garde la machinerie linéaire, mais on lui donne des variables au carré, au cube, etc.

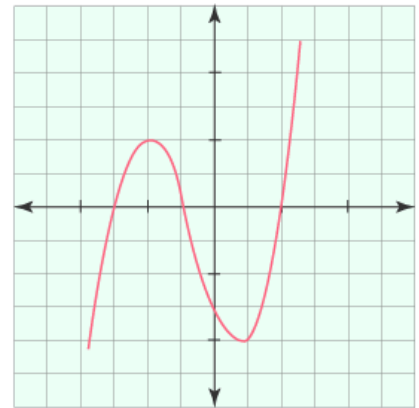
Graphs of Polynomial Functions



$$y = ax + b$$



$$y = ax^2 + bx + c$$



$$y = ax^3 + bx^2 + cx + d$$

L'essentiel

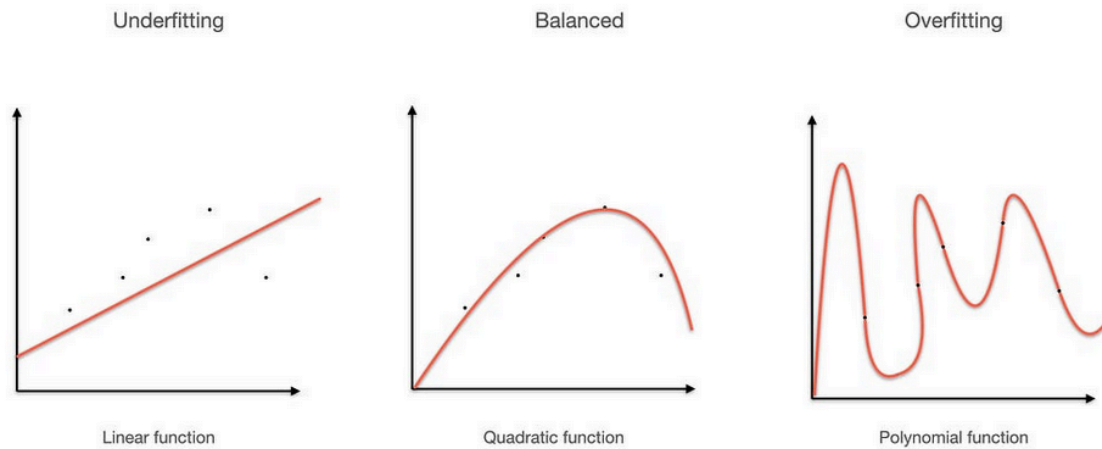
La **régression polynomiale**, c'est de la régression linéaire sur des variables transformées. Au lieu de $y = a \cdot x + b$, on fait $y = a \cdot x + b \cdot x^2 + c \cdot x^3 + \dots + d$. Le modèle reste "linéaire dans les coefficients", mais la courbe qu'il trace peut onduler.

Le danger arrive vite. Avec un degré trop élevé, la courbe passe par **tous** les points d'entraînement, y compris le bruit. Elle colle parfaitement au train et se vautre sur le test. C'est le **surapprentissage** (overfitting).

Overfitting, underfitting ?

Underfitting = le modèle est trop simple, il rate la structure (une droite sur un U).

Overfitting = le modèle est trop complexe, il apprend le bruit par coeur et ne généralise pas. On cherche le juste milieu.



Voici ce qu'on va faire dans le code qui suit, pour voir l'overfitting se produire sous nos yeux :

- fabriquer une vraie relation non linéaire (une vague, un sinus) et y ajouter un peu de bruit aléatoire,
- séparer en train et test,
- entraîner trois modèles polynomiaux de degrés croissants : 1 (une droite), 4 (une courbe douce), 15 (une courbe qui peut s'affoler),
- pour chacun, comparer le R2 sur le train et sur le test.

L'astuce pédagogique :

C'est l'**écart** entre R2 train et R2 test qui trahit l'overfitting. Un modèle qui cartonne sur le train mais s'écroule sur le test a appris le bruit par coeur.

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

# On fabrique une vraie relation non linéaire (une vague) + du bruit
rng = np.random.RandomState(42)
X = np.sort(rng.uniform(-3, 3, 80)).reshape(-1, 1)
y = np.sin(X).ravel() + rng.normal(0, 0.2, 80)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

for degre in [1, 4, 15]:
    # PolynomialFeatures fabrique x, x^2, x^3... jusqu'au degré demandé
    modele = make_pipeline(PolynomialFeatures(degre), LinearRegression())
    modele.fit(X_train, y_train)
    r2_train = r2_score(y_train, modele.predict(X_train))
    r2_test = r2_score(y_test, modele.predict(X_test))
    print(f"Degré {degre:>2} : R2 train = {r2_train:.3f} | R2 test = {r2_test:.3f}")
```

Devrait afficher quelque chose comme :

```
Degré 1 : R2 train = 0.012 | R2 test = -0.089
Degré 4 : R2 train = 0.905 | R2 test = 0.879
Degré 15 : R2 train = 0.943 | R2 test = 0.612
```

Et c'est quoi `.ravel()` ?

`numpy.ravel()` is a **function in Python** that flattens a multi-dimensional array into a one-dimensional array. It can return a view of the original array when possible, meaning changes to the flattened array may affect the original.

([source codecademy](#))

Lisez le tableau de gauche à droite.

- Degré 1 (la droite) : nul partout, underfitting.
- Degré 4 : excellent et stable entre train et test, c'est le sweet spot.
- Degré 15 : meilleur sur le train, mais **le test s'effondre** : c'est l'overfitting en flagrant délit. Le modèle a appris le bruit.

La régularisation : brider le modèle pour qu'il ne surapprenne pas.

L'idée : on punit le modèle quand ses coefficients deviennent trop gros (gros coefficients = courbe qui s'affole).

Avant d'aller plus loin, deux noms reviennent partout en ML et il faut les démystifier : **L1** et **L2**. Ce sont juste deux façons de mesurer "la taille" d'un ensemble de coefficients, donc deux façons de mesurer combien on va punir le modèle.

L1 et L2, c'est quoi au juste ?

- **L2** : on additionne les coefficients **au carré**. Comme on met au carré, un gros coefficient coûte énormément ($10 \text{ au carré} = 100$), donc L2 déteste les valeurs extrêmes et préfère plein de petits coefficients équilibrés.
- **L1** : on additionne les **valeurs absolues** des coefficients (sans carré). Du coup L1 ne s'acharne pas autant sur les gros, mais elle a une propriété magique : elle pousse carrément certains coefficients **pile à zéro**.

Ces deux mesures donnent les deux régularisations classiques :

Instant Notions : Ridge, LASSO, ElasticNet

- **Ridge (pénalité L2)** : rétrécit tous les coefficients vers zéro sans jamais les annuler. Bon quand toutes les variables comptent un peu.
- **LASSO (pénalité L1)** : met certains coefficients à zéro, donc il fait une **sélection de variables automatique** (il vire les inutiles tout seul). Bon quand on soupçonne que beaucoup de variables ne servent à rien.
- **ElasticNet** : un mélange des deux, pour avoir le beurre et l'argent du beurre.

Toutes les deux se règlent avec un seul paramètre, `alpha`, la force de la punition. C'est lui qu'on va manipuler juste après pour sentir son effet.

```
from sklearn.linear_model import Ridge, Lasso

# Sur le polynôme de degré 15 qui surapprenait, on ajoute une régularisation Ridge
modele_ridge = make_pipeline(PolynomialFeatures(15), Ridge(alpha=1.0))
modele_ridge.fit(X_train, y_train)
print(f"Ridge degré 15 : R2 test = {r2_score(y_test, modele_ridge.predict(X_test)):.3f}")
```

Devrait afficher quelque chose comme :

```
Ridge degré 15 : R2 test = 0.851
```

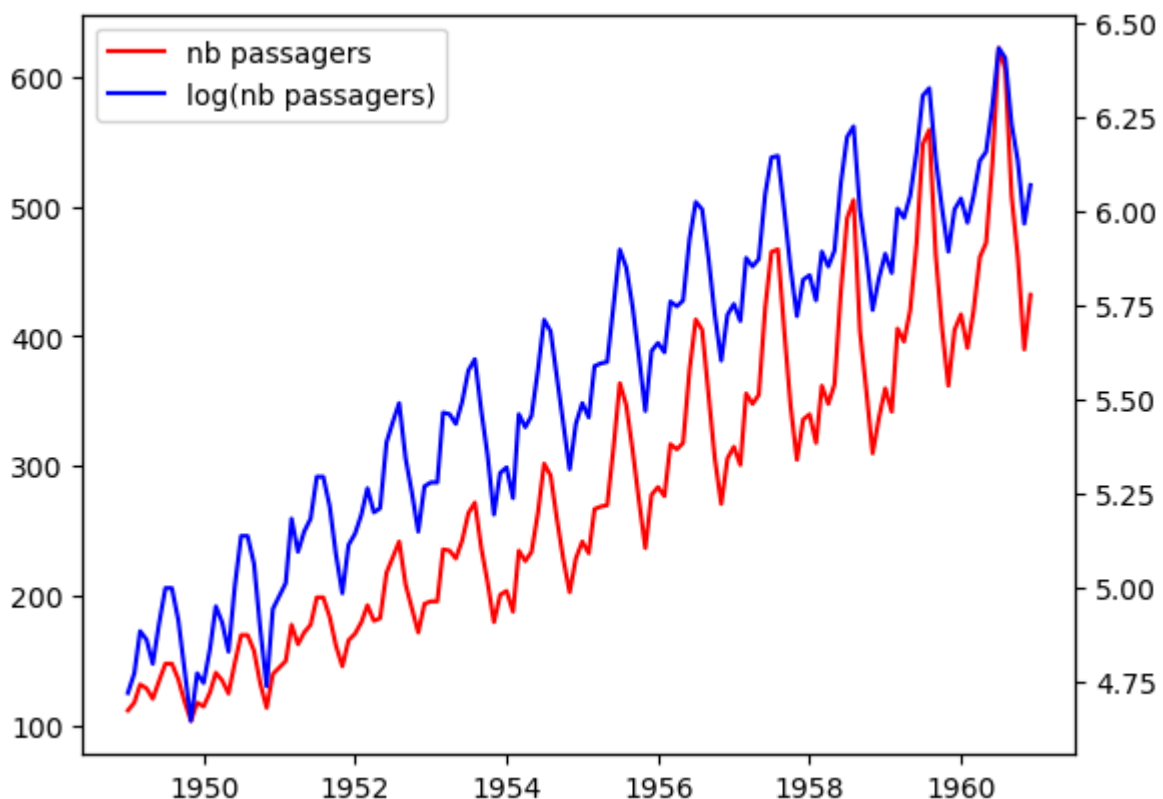
Le degré 15 qui s'effondrait à 0.61 remonte à 0.85 une fois bridé. La régularisation a sauvé le modèle de lui-même.

On bouge un paramètre : faisons varier `alpha` dans le Ridge, `0.001` puis `1.0` puis `1000`, et observons. C'est le moment de sentir ce que fait vraiment ce réglage.

On va remarquer deux choses. À `alpha = 0.001`, on bride à peine : on retombe presque sur le degré 15 qui surapprenait, donc R2 test médiocre. À `alpha = 1000`, on bride à mort : le modèle devient tellement plat qu'il sous-apprend (underfitting), et le R2 test redescend aussi. Entre les deux, il y a un `alpha` idéal, et c'est exactement pour ça qu'on le règle au lieu de le laisser au hasard. Le R2 dessine une bosse : mauvais aux extrêmes, bon au milieu. En remplaçant `Ridge` par `Lasso(alpha=0.01)`, on retrouve la même logique, avec en prime des coefficients carrément annulés.

Approfondissons

Un mot sur les **séries temporelles**, parce que vous allez en croiser et c'est un piège classique. Une série temporelle, c'est de la donnée indexée par le temps : le cours d'une action, la météo jour après jour, les ventes mensuelles. La particularité : **l'ordre compte**, et demain dépend d'aujourd'hui.



Pourquoi c'est un piège ? Décortiquons-le, parce que c'est l'erreur numéro un sur les séries temporelles, et elle est sournoise.

Le réflexe normal, c'est le `train_test_split` aléatoire de J2 : on tire 80% des lignes au hasard pour le train, 20% pour le test. Sauf que "au hasard" mélange les dates. On se retrouve à entraîner sur le mardi et le jeudi pour prédire le mercredi, qui est pile entre les deux.

Et là, on triche sans s'en rendre compte. Prédire le mercredi alors que le modèle a déjà vu le jeudi, c'est lui donner un coup d'oeil sur l'avenir. En vrai, le mercredi, le jeudi n'existe pas encore : il est impossible à connaître. On appelle ça une **fuite de données** (data leakage) : de l'information du futur qui s'infiltre dans l'entraînement.

La conséquence concrète : votre modèle affiche un score génial en test (il a triché), puis s'écroule en production sur de vraies données futures qu'il n'a jamais pu anticiper. Le piège classique du débutant en finance ou en prévision de ventes.

La règle est donc simple : sur une série temporelle, on coupe **dans le temps**. Le train, c'est tout le passé ; le test, c'est toujours la période la plus récente, d'un seul bloc, jamais un échantillon tiré au hasard. On entraîne sur janvier-octobre, on teste sur novembre-décembre, exactement comme la réalité se présentera.

Pour nous, l'essentiel à retenir tient en une astuce. On peut souvent transformer une série temporelle en problème de régression classique, juste en créant des variables "décalées", les **lag features**.

Concrètement, on rajoute des colonnes du genre : "la valeur d'hier", "la valeur d'il y a 7 jours", "la moyenne des 30 derniers jours". Chaque ligne emporte ainsi son petit bout de passé.

Une fois ces colonnes créées, n'importe quel algo de régression de la journée peut tourner dessus, sans rien changer d'autre. C'est bluffant de simplicité.

Les modèles vraiment dédiés au temps (comme [ARIMA](#) ou Prophet, deux outils spécialisés, et les réseaux récurrents que vous verrez en 5e année) sont hors programme ici. Mais l'astuce des lag features, elle, est un vrai outil de terrain qui vous dépannera souvent.

ARIMA : ARIMA stands for Autoregressive Integrated Moving Average, which is a statistical model used for analyzing and forecasting time series data. It combines autoregressive and moving average components while addressing non-stationarity through differencing, making it a popular choice for predicting future values based on past observations.

Pour creuser un peu ces notions de deep learning, [rendez-vous ici](#).

On maîtrise la régression et comment la brider. Mais jusqu'ici on prédit des nombres. Et si on voulait prédire une catégorie : spam ou pas spam, malade ou sain ? Bascule vers la classification, avec un algo dont le nom trompe tout le monde.

5 - Régression logistique et scoring

Piège de vocabulaire d'entrée : la **régression** logistique fait de la **classification**. Oui, le nom est trompeur, tout le monde se fait avoir une fois. Scénario concret : une banque veut décider d'accorder un crédit. Elle ne veut pas un "oui/non" brut, elle veut une **probabilité de défaut** : "ce client a 12% de chances de ne pas rembourser". Avec cette proba, elle fixe son propre seuil selon son appétit pour le risque. C'est exactement le scoring, et c'est le métier de la régression logistique.

L'essentiel

La **régression logistique** prédit une probabilité entre 0 et 1. Sous le capot, elle calcule un score linéaire (comme la régression linéaire), puis l'écrase entre 0 et 1 avec une fonction en S, la **sigmoïde**.

La sigmoïde ? Une fonction qui transforme n'importe quel nombre en une valeur entre 0 et 1. Très négatif => proche de 0. Très positif => proche de 1. Zéro => 0.5 pile. C'est ce qui transforme un score brut en probabilité. (Vous la recroiserez en J4, c'est aussi une fonction d'activation des réseaux de neurones.)

Pour décider, on applique un **seuil** : par défaut, $\text{proba} > 0.5 \Rightarrow$ classe 1. Mais ce seuil, c'est VOUS qui le choisissez, et c'est tout le levier métier.

stratify=y, ça sert à quoi ? Ça garde la même proportion de classes dans le train et dans le test. Sans ça, si les spams sont rares, le test pourrait n'en contenir presque aucun. Indispensable dès qu'une classe est minoritaire.

Et max_iter ? C'est le nombre maximum d'étapes que l'algo s'autorise pour converger (trouver la meilleure solution et se stabiliser). Trop petit : il s'arrête trop tôt et scikit-learn lève un warning. 5000 est confortable pour la plupart des jeux.

Voici le déroulé du code qui suit, sur un dataset de tumeurs (maligne ou bénigne) :

- charger les 569 patients et leurs mesures,
- séparer train/test en gardant la proportion de classes (`stratify`),
- standardiser les données (la logistique converge mal sinon, réflexe J2),
- entraîner la régression logistique,
- récupérer non pas la décision brute mais la **probabilité** de chaque cas (`predict_proba`),
- enfin décider avec le seuil par défaut de 0.5 et lire le rapport complet.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
import numpy as np

# 569 patients, tumeur maligne (0) ou bénigne (1)
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

# La logistique a besoin de données mises à l'échelle pour bien converger (vu en J2)
scaler = StandardScaler().fit(X_train)
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)

modele = LogisticRegression(max_iter=5000)
modele.fit(X_train_s, y_train)

# predict_proba renvoie la probabilité de chaque classe, pas juste la décision
proba = modele.predict_proba(X_test_s)[: , 1] # proba de la classe 1 (bénigne)
print("5 premières probabilités :", proba[:5].round(2))
```

```
# Décision au seuil par défaut de 0.5
y_pred = (proba >= 0.5).astype(int)
print(classification_report(y_test, y_pred, zero_division=0))
```

Devrait afficher quelque chose comme :

```
5 premières probabilités : [1.    0.99 0.04 1.    0.99]
      precision    recall  f1-score   support
0         0.98      0.93      0.95         42
1         0.96      0.99      0.97         72
accuracy                  0.96         114
```

Le modèle ne dit pas juste "bénigne" ou "maligne", il dit **à quel point il est sûr**. Et c'est là que le seuil entre en jeu.

Manip express : faisons bouger le seuil, 0.3 puis 0.5 puis 0.7, et regardons l'effet sur la classe 0 (maligne, celle qu'on ne veut surtout pas rater) :

```
for seuil in [0.3, 0.5, 0.7]:
    y_pred = (proba >= seuil).astype(int)
    # On regarde le recall de la classe 0 (maligne) : combien de cancers on attrape
    from sklearn.metrics import recall_score
    rec = recall_score(y_test, y_pred, pos_label=0)
    print(f"Seuil {seuil} : recall sur les tumeurs malignes = {rec:.2%}")
```

On va remarquer un effet de bascule super important. En baissant le seuil, on devient plus "méfiant" : on déclare maligne au moindre doute, donc on attrape **plus** de cancers (recall qui monte). Génial ? Pas gratuitement : on déclenche aussi plus de fausses alertes sur des tumeurs en réalité bénignes. C'est le curseur precision/recall de la section 2, en chair et en os. Et le bon seuil n'est pas une question de maths, c'est une décision médicale : pour un dépistage, on accepte plus de fausses alertes pour ne rater aucun vrai cancer, donc on baisse le seuil.

Pour les curieux

La régression logistique est le cheval de bataille du **credit scoring** depuis des décennies, et elle n'a pas été détrônée par hasard.

Son atout n'est pas la performance pure, c'est qu'elle est **réglementaire**. En finance, une décision de refus de crédit doit pouvoir être expliquée au client et à un régulateur : "votre demande est refusée car votre taux d'endettement pèse négativement".

La logistique donne ça directement via ses coefficients, qu'on transforme en **score de points** (le fameux "score FICO" aux États-Unis fonctionne sur ce principe : chaque facteur ajoute ou retire des points).

Un Gradient Boosting serait peut-être 2% plus précis, mais inexploitable juridiquement : impossible d'expliquer proprement sa décision. Encore un cas où le modèle le plus simple gagne, pour des raisons qui n'ont rien à voir avec la précision pure.

On sait classer avec des étiquettes. Maintenant on change complètement de monde : et si on n'avait AUCUNE étiquette ? Comment trouver des groupes que personne n'a définis ? C'est le seul détour non-supervisé de la journée, et il sert directement le TP AirBnB de cet après-midi.

6 - Clustering : KMeans et classification hiérarchique

Scénario : Airbnb a des milliers d'annonces, sans aucune catégorie prédéfinie. Personne n'a jamais étiqueté "logement premium", "petit budget", "familial". Mais ces groupes existent sûrement dans les données. Comment les faire émerger automatiquement, sans rien savoir à l'avance ? C'est le clustering, le coeur du non-supervisé.

L'essentiel

Le **clustering** regroupe les points qui se ressemblent, sans aucune étiquette. L'algo star, c'est **KMeans**.

Comment marche KMeans ? On lui dit "trouve-moi k groupes" (par exemple 3). Il place 3 centres au hasard, assigne chaque point au centre le plus proche, recalcule les centres au milieu de leurs points, et recommence jusqu'à stabilisation. Résultat : k groupes compacts.



Le hic : il faut choisir k à l'avance. Et c'est non-supervisé, donc pas d'accuracy pour trancher. On a trois notions à bien séparer pour choisir k . Prenons-les dans l'ordre, parce que la deuxième s'appuie sur la première.

Notion 1, l'inertie. C'est la somme des distances de chaque point au centre de son groupe.

Concrètement, ça mesure à quel point les groupes sont **compacts** : inertie basse = points bien serrés autour de leur centre. Petit piège : l'inertie baisse toujours quand on ajoute des clusters (avec un cluster par point, elle tombe à zéro), donc on ne peut pas juste prendre le k qui minimise l'inertie. Sinon on aurait mille groupes d'un point chacun.

Notion 2, la méthode du coude (elbow). Justement pour contourner ce piège. On trace l'inertie en fonction de k et on regarde la **forme** de la courbe. Au début, ajouter un cluster fait chuter l'inertie d'un coup (on sépare des groupes vraiment distincts). Passé le vrai nombre de groupes, ajouter un cluster ne gagne presque plus rien : la courbe s'aplatit. Ce point de cassure, là où ça passe de "chute forte" à "ça stagne", dessine un coude. Ce coude est souvent le bon k.

Notion 3, le score de silhouette. Une mesure plus directe, entre -1 et 1. Pour chaque point, elle compare sa distance à ses voisins de groupe (doit être petite) et sa distance au groupe voisin le plus proche (doit être grande). Proche de 1 = le point est pile dans le bon groupe. Proche de 0 = il est à la frontière. Négatif = il est probablement dans le mauvais groupe. On prend le k qui **maximise** la silhouette moyenne.

Mettons ces trois notions au travail. Le code va :

- fabriquer 4 groupes bien séparés (on connaît la réponse, c'est pour vérifier que les méthodes la retrouvent),
- tester plusieurs valeurs de k, de 2 à 6,
- pour chaque k, lancer KMeans et relever **inertie** et **silhouette**,
- afficher les deux côte à côte pour repérer le coude et le pic de silhouette.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# On fabrique 4 groupes bien séparés pour l'exemple
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=42)

# On teste plusieurs k et on regarde inertie + silhouette
for k in [2, 3, 4, 5, 6]:
    # n_init=10 : KMeans relance 10 fois avec des départs différents et garde le meilleur
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    sil = silhouette_score(X, km.labels_)
    print(f"k={k} : inertie = {km.inertia_:7.1f} | silhouette = {sil:.3f}")
```

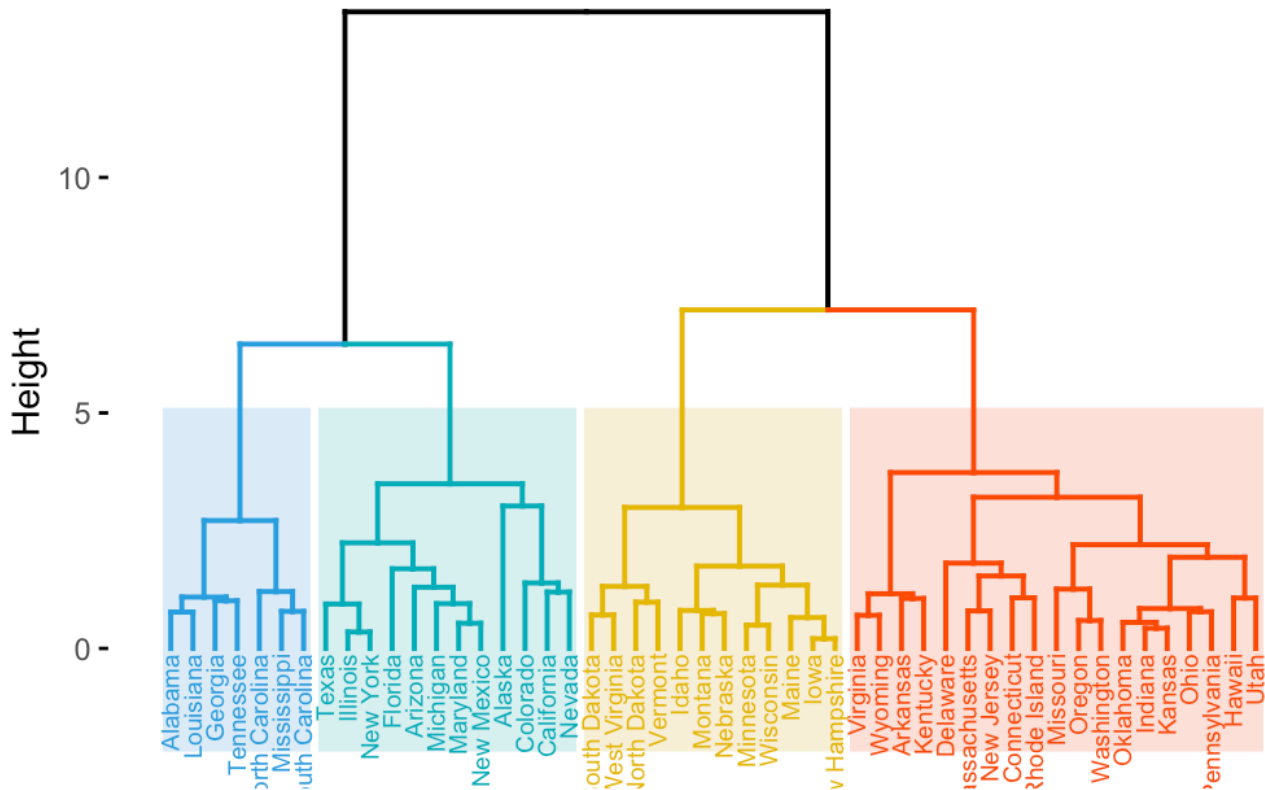
Devrait afficher quelque chose comme :

```
k=2 : inertie = 3232.6 | silhouette = 0.578
k=3 : inertie = 1521.4 | silhouette = 0.668
k=4 : inertie = 411.9 | silhouette = 0.792
k=5 : inertie = 362.1 | silhouette = 0.639
k=6 : inertie = 318.4 | silhouette = 0.573
```

L'inertie chute fort jusqu'à k=4 puis ralentit (le coude), et la silhouette est maximale à k=4. Les deux méthodes sont d'accord : il y a bien 4 groupes. Logique, on en a fabriqué 4.

L'autre famille : la classification hiérarchique. Au lieu de fixer k à l'avance, elle construit un arbre de regroupements (un **dendrogramme**) : chaque point commence seul, puis on fusionne les plus proches, étape par étape, jusqu'à un seul gros groupe. On choisit ensuite où "couper" l'arbre pour avoir le nombre de groupes voulu.

Cluster Dendrogram



KMeans vs hiérarchique, lequel ? KMeans est rapide et passe sur de gros volumes, mais exige k à l'avance et suppose des groupes ronds. La hiérarchique ne demande pas k au départ et donne un bel arbre interprétable, mais elle est lente sur beaucoup de données.

=> KMeans pour le volume,

=> hiérarchique pour comprendre la structure.

On touche le terrain : changeons le nombre de vrais groupes générés, `centers=4` devient `centers=6`, et relançons. On va remarquer que le coude et le pic de silhouette se déplacent proprement vers $k=6$: les méthodes retrouvent la vérité qu'on a cachée. Puis poussons `cluster_std` à `3.0` pour que les groupes se chevauchent : là c'est plus flou, la silhouette baisse et le coude devient mou. C'est la leçon : sur des groupes nets, ces méthodes sont catégoriques ; sur des groupes qui se mélangent, elles hésitent, et c'est normal, parce que la "vérité" elle-même est devenue floue.

On va plus loin

KMeans a trois angles morts qu'il faut connaître avant de l'utiliser en TP cet après-midi.

Angle mort 1 : il suppose des clusters sphériques et de taille comparable. Sur des groupes allongés ou imbriqués (imaginez deux croissants de lune entrelacés), il se plante complètement, parce qu'il raisonne en "distance au centre" et un croissant n'a pas de centre qui le résume.

Angle mort 2 : il est sensible à l'échelle. Si une colonne va de 0 à 1 et une autre de 0 à 1 000 000, la seconde écrase tout le calcul de distance. **Il faut TOUJOURS standardiser avant un KMeans** (le `StandardScaler` de `J2`), sinon le résultat n'a aucun sens. Retenez bien ça, c'est exactement le piège qu'on vous tend cet après-midi sur le TP AirBnB.

Angle mort 3 : il est sensible au point de départ aléatoire. Selon où tombent les centres initiaux, il peut converger vers un mauvais découpage. D'où le `n_init=10` : il relance 10 fois avec des départs différents et garde le meilleur.

Et pour les clusters non sphériques, il existe **DBSCAN**, un autre algo qui trouve même le nombre de groupes tout seul et repère les points isolés. À garder dans un coin de la tête pour vos projets.

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) est l'un des algorithmes de clustering les plus puissants du machine learning non supervisé. Contrairement au K-Means qui impose des clusters sphériques et exige de connaître à l'avance leur nombre, DBSCAN découvre automatiquement des clusters de forme quelconque en se basant sur la densité locale des données.

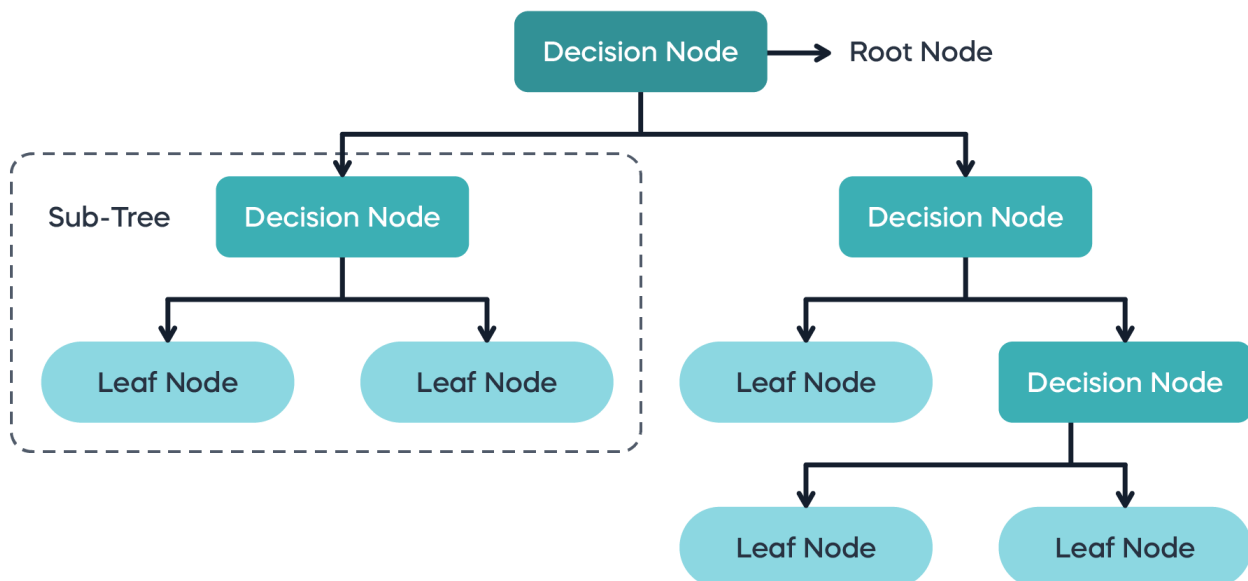
[Guide ici](#)

On a couvert régression et clustering. On passe à une famille d'algos qui pense comme un humain qui pose des questions : les arbres. Et avec eux, un second algo, simpliste sur le papier mais redoutable sur le texte, qui va nous servir pour le TP spam.

7 - Arbres de décision et Naive Bayes

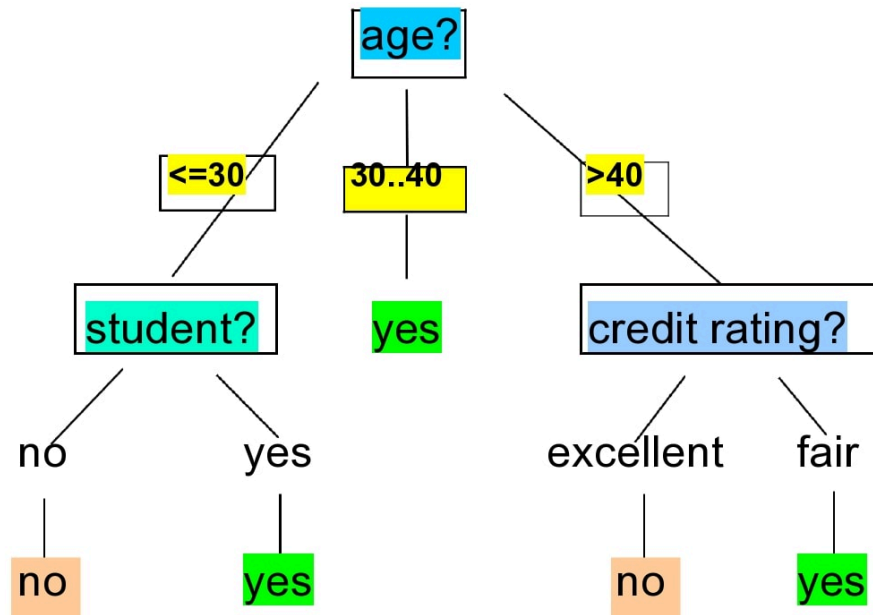
Comment un médecin pose-t-il un diagnostic ? Il enchaîne des questions : "fièvre ? oui. Toux ? oui. Depuis plus de 3 jours ? non." À chaque réponse, il élague les possibilités. Un **arbre de décision** fait exactement pareil, sauf que c'est l'algo qui trouve tout seul les meilleures questions à poser.

Le principe :



Avec exemple :

Output: A Decision Tree for



L'essentiel des arbres et de Naive Bayes

Un **arbre de décision** découpe les données par des questions successives sur les variables. Chaque noeud pose une question ("surface > 80 m2 ?"), chaque branche est une réponse, chaque feuille est une prédiction.

Comment l'arbre choisit-il ses questions ? À chaque étape, il cherche la coupure qui sépare le mieux les classes. Reste à définir "le mieux", et c'est là qu'arrivent deux mots qu'on entend tout le temps sans toujours savoir ce qu'ils veulent dire : **Gini** et **entropie**. Définissons-les proprement, ce sont juste deux façons de mesurer le **mélange** dans un groupe.

L'impureté de Gini, c'est quoi ? La probabilité de se tromper si on étiquetait un point au hasard selon la composition du groupe. Un groupe 100% spam : aucun risque de se tromper, Gini = 0 (groupe **pur**). Un groupe moitié spam moitié normal : une chance sur deux de se tromper, Gini = 0.5 (mélange maximal). Plus c'est bas, plus le groupe est homogène.

Et l'entropie ? La même idée, venue de la théorie de l'information : elle mesure le "désordre" du groupe. Groupe pur = entropie 0 (aucune surprise, on sait ce qu'on va tirer). Groupe parfaitement mélangé = entropie maximale (surprise totale). En pratique, Gini et entropie donnent des arbres quasi identiques ; Gini est juste un poil plus rapide à calculer, c'est le défaut de scikit-learn.

L'arbre teste donc toutes les coupures possibles et garde celle qui fait **le plus chuter** cette impureté : celle qui rend les deux groupes enfants les plus purs possible. Il répète, encore et encore, jusqu'aux feuilles.

Voici ce que le code va montrer, sur le dataset des fleurs iris (3 espèces à reconnaître) :

- charger les données et entraîner un arbre, en limitant sa profondeur à 3 pour qu'il reste lisible,

- mesurer son accuracy sur le test,
- et surtout **afficher l'arbre en texte**, pour lire l'enchaînement de questions qu'il a inventé tout seul.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
noms = load_iris().feature_names
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

# max_depth=3 : on limite la profondeur pour garder l'arbre lisible (et éviter
l'overfitting)
arbre = DecisionTreeClassifier(max_depth=3, random_state=42)
arbre.fit(X_train, y_train)
print(f"Accuracy : {accuracy_score(y_test, arbre.predict(X_test)):.2%}")

# On peut LIRE l'arbre en texte : c'est l'enchaînement de questions
print(export_text(arbre, feature_names=noms))
```

Devrait afficher quelque chose comme :

```
Accuracy : 93.33%
|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) > 0.80
|   |--- petal width (cm) <= 1.75
|       |--- class: 1
|       |--- petal width (cm) > 1.75
|           |--- class: 2
```

L'énorme atout : c'est **lisible par un humain**. On suit les branches comme un organigramme. Le gros défaut : un arbre seul **surapprend facilement**. Si on ne limite pas sa profondeur, il finit par poser une question par exemple d'entraînement et apprend tout par coeur. D'où le `max_depth`. (La section suivante règle ce défaut avec les forêts.)

Le second algo : Naive Bayes. Complètement différent. Il repose sur les probabilités (le théorème de Bayes) et une hypothèse "naïve" : il suppose que toutes les variables sont indépendantes. C'est faux dans la vraie vie, mais ça marche étonnamment bien, surtout sur le texte.

Pourquoi "naïf", et pourquoi ça brille sur le texte ? "Naïf" parce qu'il suppose que la présence du mot "gratuit" et celle du mot "urgent" sont indépendantes dans un mail (en vrai elles vont souvent ensemble). Mais sur du texte, où chaque mot est une variable, cette simplification le rend ultra-rapide et étonnamment précis. C'est l'algo historique de la détection de spam, et c'est exactement ce qu'on utilisera cet après-midi pour le TP courriel vs spam.

Le code qui suit va, sur un mini jeu de 4 messages seulement :

- transformer chaque message en comptage de mots (le **bag of words**, défini juste après),

- entraîner Naive Bayes là-dessus,
- puis lui soumettre un message jamais vu et lui demander sa probabilité de spam.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Mini jeu de texte : 4 messages, étiquetés spam (1) ou normal (0)
messages = ["gagnez de 1 argent gratuit maintenant", "reunion demain a 14h salle 3",
            "offre gratuite urgent cliquez ici", "peux tu relire mon rapport stp"]
labels = [1, 0, 1, 0]

# CountVectorizer transforme le texte en tableau de comptage de mots (bag of words)
vec = CountVectorizer()
X = vec.fit_transform(messages)

modele = MultinomialNB().fit(X, labels)

# On teste sur un message jamais vu
test = vec.transform(["argent gratuit urgent"])
print("Probabilité spam :", modele.predict_proba(test)[0][1].round(3))
print("Décision (1=spam) :", modele.predict(test)[0])
```

Devrait afficher quelque chose comme :

```
Probabilité spam : 0.92
Décision (1=spam) : 1
```

Avec 4 exemples seulement, Naive Bayes repère déjà que "argent gratuit urgent" sent le spam à plein nez. C'est sa magie : peu de données, traitement quasi instantané du texte.

C'est quoi un "bag of words" ? Une façon de transformer du texte en chiffres : on compte combien de fois chaque mot apparaît, sans tenir compte de l'ordre. "le chat dort" et "dort le chat" donnent le même vecteur. Simpliste, mais redoutablement efficace pour le spam.

On bouge un paramètre : dans le code de l'arbre, retirons le `max_depth=3` pour le laisser grandir sans limite, et comparons l'accuracy sur le train et sur le test. On va remarquer que le train grimpe souvent à 100% pendant que le test, lui, plafonne ou baisse : c'est l'overfitting en flagrant délit, l'arbre a posé une question par exemple jusqu'à tout apprendre par coeur. Côté Naive Bayes, glissons un message ambigu dans `test`, genre "reunion gratuite demain" (un mot de spam, "gratuite", noyé dans un message normal) : on verra sa probabilité de spam se rapprocher de 0.5, sa façon honnête de dire "là, je ne sais pas trancher".

Creusons

Petite confusion fréquente à lever : les arbres et Naive Bayes répondent à la même tâche (classification) mais raisonnent à l'opposé. L'arbre **partitionne l'espace** par des coupures nettes (des "si/alors"), il gère bien les interactions entre variables ("grande surface ET centre-ville"). Naive Bayes **multiplie des probabilités** et suppose justement qu'il n'y a pas d'interaction. Conséquence pratique : sur des données tabulaires avec des effets croisés, l'arbre (et surtout les forêts qu'on voit juste après) écrase Naive Bayes. Sur du texte avec des milliers de mots-variables, Naive Bayes reste imbattable en rapport simplicité/performance. Le bon algo dépend de la **forme de la donnée**, jamais d'un classement absolu. C'est le fil rouge de toute la journée.

On a notre premier classifieur lisible, mais fragile seul. L'idée géniale de la suite : et si au lieu d'un arbre, on en faisait voter des centaines ? C'est le saut vers les modèles qui gagnent la plupart des compétitions sur données tabulaires.

8 - Random Forest et Gradient Boosting

Un arbre seul, c'est un expert ombrageux : il a des avis tranchés et il se trompe parfois lourdement (overfitting). Et si, plutôt que de croire un seul expert, on demandait à 300 experts de voter ? C'est l'intuition derrière les **méthodes d'ensemble**, et c'est ce qui domine les compétitions [Kaggle](#) (la plateforme de compétitions de data science) sur données tabulaires depuis 15 ans.

L'essentiel du bagging et du boosting

Il y a deux grandes façons de combiner plein d'arbres, et il faut les distinguer nettement (question d'entretien quasi garantie).

Instant Notions : bagging vs boosting

- **Bagging (Random Forest)** : on entraîne des centaines d'arbres **en parallèle**, chacun sur un échantillon aléatoire des données et des variables. Puis on fait voter tout le monde. Chaque arbre est imparfait, mais leurs erreurs se compensent. Effet : ça **réduit la variance** (le modèle est plus stable, moins sensible au bruit).
- **Boosting (Gradient Boosting)** : on entraîne les arbres **en séquence**, et chaque nouvel arbre se concentre sur les erreurs du précédent. On corrige progressivement. Effet : ça **réduit le biais** (le modèle devient très précis), au risque de surapprendre si on en met trop.
- **Le slogan** : le bagging fait voter des experts indépendants ; le boosting fait se relayer des experts qui réparent les bourdes du précédent.

C'est quoi la variance et le biais ? Le biais, c'est l'erreur systématique (le modèle est trop simple, il rate la cible toujours du même côté). La variance, c'est l'instabilité (le modèle change beaucoup si on change un peu les données). On cherche à minimiser les deux, mais réduire l'un augmente souvent l'autre : c'est le compromis biais-variance.

Mettons les trois face à face sur le même problème (détection de tumeur). Le code va :

- charger les données et faire un seul découpage train/test, commun aux trois,
- monter trois modèles dans un dictionnaire : un arbre seul, une Random Forest (200 arbres), un Gradient Boosting,
- les entraîner et afficher leur accuracy côte à côte, pour voir l'écart entre l'arbre solitaire et les ensembles.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
```

```
modeles = {
    "Arbre seul": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
}
for nom, m in modeles.items():
    m.fit(X_train, y_train)
    acc = accuracy_score(y_test, m.predict(X_test))
    print(f"{nom:>18} : {acc:.2%}")
```

Devrait afficher quelque chose comme :

```
Arbre seul : 94.74%
Random Forest : 96.49%
Gradient Boosting : 95.61%
```

L'arbre seul est le plus faible, la forêt et le boosting le dépassent en faisant voter ou se relayer beaucoup d'arbres. Sur ce jeu c'est serré, sur des problèmes plus durs l'écart se creuse souvent nettement en faveur des ensembles.

Aparté : l'importance des variables. Les forêts offrent un bonus précieux :

`modele.feature_importances` dit quelles variables ont le plus pesé dans les décisions. C'est un excellent outil pour comprendre son problème et virer les variables inutiles.

Manip express : faisons varier `n_estimators`, le nombre d'arbres de la forêt, en testant 1, 10 puis 500. On va remarquer un truc rassurant : entre 1 et 10 arbres, l'accuracy bondit, mais entre 200 et 500, le gain devient quasi nul. C'est la signature du bagging : passé un certain nombre de votants, ajouter des arbres ne sert plus à rien, on a déjà lissé le bruit. Côté Gradient Boosting, en passant `learning_rate=0.01` (au lieu de 0.1) avec `n_estimators=500`, on apprend plus lentement mais plus finement : avec assez d'arbres, un petit pas d'apprentissage donne souvent un modèle plus robuste qu'un grand pas pressé.

Pour aller plus loin

Dans la vraie vie et sur Kaggle, le boosting maison de scikit-learn est rarement celui qu'on utilise.

Les vraies références sont [XGBoost](#), [LightGBM](#) et [CatBoost](#) : trois bibliothèques de Gradient Boosting ultra-optimisées qui raflent une bonne partie des compétitions sur données tabulaires.

Elles font la même chose que `GradientBoostingClassifier`, mais en 100 fois plus rapide et avec de meilleurs réglages par défaut. Si un jour vous attaquez une compétition Kaggle tabulaire, c'est par LightGBM ou XGBoost que vous commencerez.

Pour aujourd'hui, le `GradientBoostingClassifier` de scikit-learn suffit largement : il illustre le concept, et c'est le concept qui compte. La leçon de fond tient toujours : un ensemble bien réglé bat presque toujours un modèle unique.

À mater ce soir : [StatQuest - Random Forests](#) (~17 min) : la meilleure explication visuelle de comment une forêt bat un arbre seul, avec des dessins clairs. Enchaînez avec [StatQuest - Gradient Boost Part 1](#) (~17 min) pour voir en images comment le boosting corrige ses erreurs séquentiellement.

On a les algos qui gagnent les compétitions. Reste un dernier combattant, longtemps roi avant les forêts, avec une idée géométrique élégante et un tour de magie pour gérer le non-linéaire : le SVM. Après lui, on saura choisir quel algo dégainer en premier.

9 - [SVM](#) et la règle "quel algo en premier"

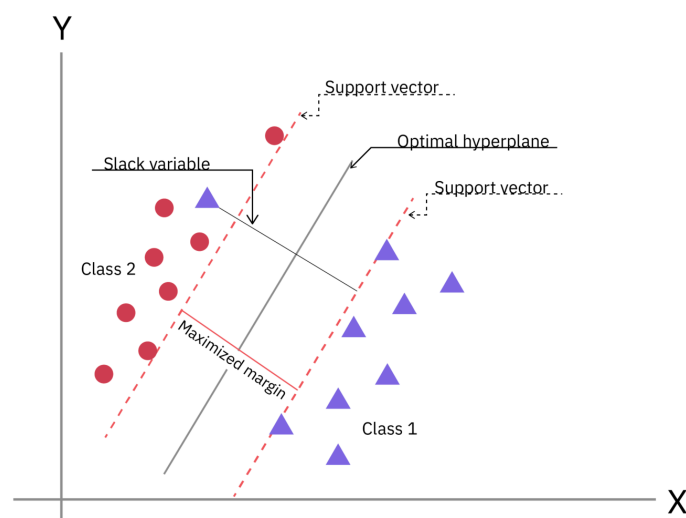
Imaginez deux nuages de points sur une feuille, l'un rouge, l'autre bleu, qu'on veut séparer par une ligne. Il y a une infinité de lignes possibles. *Laquelle est la meilleure ?* Pour un **SVM**, c'est celle qui laisse le plus grand couloir vide entre les deux camps. Cette idée simple (maximiser la marge) en fait un classifieur redoutable.

L'essentiel du SVM

Le **SVM** (Support Vector Machine, machine à vecteurs de support) cherche la frontière qui sépare les classes avec la **marge la plus large possible**. Plus le couloir est large, plus le modèle est robuste sur des points nouveaux.

C'est quoi les vecteurs de support ? Les quelques points situés tout au bord de la marge, ceux qui "tiennent" la frontière. Si on les déplace, la frontière bouge. Tous les autres points, loin de la frontière, ne comptent pas. D'où le nom : seuls ces vecteurs "supportent" la décision.

SVM: Margin and Classification Explained



Mais que faire si les classes ne sont **pas séparables par une ligne droite** (deux cercles concentriques, par exemple) ? C'est là qu'intervient le tour de magie.

Le kernel trick, en intuition. Si on ne peut pas séparer en 2D, on projette les points dans une dimension supérieure où, comme par enchantement, une séparation droite devient possible. Imaginez des points rouges au centre et bleus autour, sur une table : impossible de les séparer avec une règle. Mais soulevez les rouges en l'air (ajoutez une 3e dimension) et là, une feuille horizontale les sépare net. Le "kernel" fait ce soulèvement sans jamais calculer explicitement les nouvelles coordonnées. C'est mathématiquement élégant et computationnellement malin.

Les kernels courants sont : `linear` pour une frontière droite, `rbf` le plus polyvalent (frontières courbes), et `poly` pour une approche polynomiale.

Pour voir le kernel trick en action, le code va monter un cas piège exprès :

- fabriquer deux cercles imbriqués, l'un dans l'autre, **impossibles** à séparer par une droite,
- entraîner un SVM avec le kernel `linear` (la droite) puis avec le kernel `rbf` (qui sait "soulever" les points),
- comparer leurs accuracy pour mesurer l'écart entre les deux.

```
from sklearn.datasets import make_circles
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Deux cercles imbriqués : INSÉPARABLES par une droite
X, y = make_circles(n_samples=300, noise=0.1, factor=0.4, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

for kernel in ["linear", "rbf"]:
    modele = SVC(kernel=kernel)
    modele.fit(X_train, y_train)
    acc = accuracy_score(y_test, modele.predict(X_test))
    print(f"Kernel {kernel:>6} : accuracy = {acc:.2%}")
```

Devrait afficher quelque chose comme :

```
Kernel linear : accuracy = 48.89%
Kernel      rbf : accuracy = 98.89%
```

Le kernel linéaire fait à peine mieux que pile ou face (les cercles ne sont pas séparables par une droite), mais le kernel `rbf` monte à 99% : il a "soulevé" les points dans une dimension où la séparation devient évidente. Voilà le kernel trick en action.

On touche le terrain : sur le SVC `rbf`, jouons avec le paramètre `C`, qui contrôle la tolérance aux erreurs : petit `C` = marge large mais tolérante, grand `C` = marge étroite mais stricte. En testant `C=0.01` puis `C=100`, on va remarquer qu'un `C` trop petit laisse passer trop d'erreurs (la frontière devient molle) tandis qu'un `C` trop grand colle de trop près aux points d'entraînement (un parfum d'overfitting). Le bon réglage est un compromis, comme souvent. Même histoire avec `gamma` (`gamma=10` rend la frontière très sinueuse et locale) : plus on pousse, plus le modèle épouse le moindre point, jusqu'à surapprendre.

Pour les curieux

Le SVM a un talon d'Achille pratique : il **passe mal à l'échelle**. Son temps de calcul explose quand le nombre d'exemples grandit ; au-delà de quelques dizaines de milliers de lignes, ça devient franchement lent.

C'est l'une des raisons pour lesquelles, dans les années 2010, le Gradient Boosting et les Random Forests l'ont largement détrôné sur les gros volumes tabulaires.

Mais il n'a pas dit son dernier mot : le SVM reste excellent sur des jeux de taille modeste avec beaucoup de variables (texte vectorisé, bio-informatique). Le TP sonar de cet après-midi (208 lignes, 60 variables) est précisément un terrain où il peut briller.

Comme toujours, la morale est la même : pas de meilleur algo dans l'absolu, un meilleur algo **pour une forme de problème donnée**.

Dans vos favoris : [StatQuest - Support Vector Machines](#) (~20 min) : visualisation animée de la marge, des vecteurs de support et du kernel trick, sans une seule équation qui fait peur.

La règle : quel algo essayer en premier ?

On vient de voir une dizaine d'algos. Face à un nouveau problème, par lequel commencer ? Voici l'arbre de décision mental d'un data scientist (à nuancer, mais c'est un excellent point de départ) :

Situation	Premier réflexe	Pourquoi
Prédire un nombre, besoin d'expliquer	Régression linéaire (+ Ridge/LASSO)	La baseline lisible et auditable
Prédire un nombre, relation non linéaire	Random Forest ou Gradient Boosting	Captent le non-linéaire sans réglage manuel
Classer, besoin d'une proba et d'expliquer	Régression logistique	Scoring réglementaire, seuil ajustable
Données tabulaires, on veut juste la perf	Random Forest puis Gradient Boosting	Les champions du tabulaire
Du texte (spam, sentiment)	Naive Bayes (baseline) puis logistique	Rapide et solide sur les mots
Peu de données, beaucoup de variables	SVM (kernel rbf)	Brille quand n est petit
Aucune étiquette, trouver des groupes	KMeans (après standardisation)	Le clustering de référence

La méthode de terrain : on commence **toujours par une baseline simple** (linéaire ou logistique), on note son score dans l'Arène, puis on monte en complexité **seulement si ça vaut le coup**. Un modèle compliqué qui ne bat pas la baseline ne mérite pas d'être déployé. C'est exactement ce qu'on va faire cet après-midi.

À votre tour

Tout l'arsenal est posé. Cet après-midi, vous pilotez. On attaque quatre problèmes réels, chacun d'une famille différente, puis on organise le grand combat : le **Fight des IA**. À la fin, vous avez un leaderboard d'algos comparés sur GitHub, et vous savez justifier votre champion. C'est, en miniature, exactement ce qu'on fait sur Kaggle. 🌟

Le projet

Vous construisez votre **Arène des Algos** : un notebook qui, sur quatre datasets, fait tourner et compare des algos, et culmine sur un benchmark où tous s'affrontent sur le même jeu, même découpage, même métrique. Le livrable :

- un notebook propre, une section par phase
- un tableau final (le leaderboard) qui classe les algos sur le jeu du Fight

- un README qui désigne le champion et **justifie le choix** (pas juste "il a le meilleur score", mais "voici le bon score à regarder vu le problème")

Ici, vous avez les signatures, les sorties attendues et les checkpoints. À vous d'écrire le code. Bloquer un peu, c'est le but.

Phase 0 : Mise en route

On repart du repo `arene-des-algos` de J1. Créez un dossier `j3/`.

- Ouvrez [Google Colab](#) ou votre Jupyter local.
- Vérifiez vos imports : `scikit-learn`, `pandas`, `numpy`, `matplotlib`.
- Pour le TP spam, vous aurez besoin du dataset [SMS Spam Collection \(UCI\)](#) ; pour AirBnB, d'un CSV de listings ([Inside Airbnb](#) ou un dataset Kaggle Airbnb).

Avant de coder : votre repo GitHub est votre copie d'examen continue. On note les commits, pas que le résultat final. Committez à chaque phase finie, message clair, et pas de `git add .` à l'aveugle : fichier par fichier.

Phase A : Prédire les prix immobiliers (régression)

Le problème : estimer le prix médian d'un logement en Californie à partir de variables comme le revenu médian du quartier, l'âge des logements, la population. C'est de la **régression** (on prédit un nombre).

Dataset : `fetch_california_housing` de scikit-learn (20 640 quartiers, 8 variables). On n'utilise PAS Boston Housing, retiré pour des biais éthiques dans ses variables.

Montez le pipeline : charger, split, standardiser, entraîner une régression linéaire (la baseline), puis un Random Forest, et comparer avec les 3 métriques de régression.

```
from sklearn.datasets import fetch_california_housing

def charger_immobilier():
    """Charge California Housing, renvoie X, y.
    Doit afficher : nombre de lignes, nombre de variables, et l'unité de la cible.
    """
    # TODO : data = fetch_california_housing()
    # TODO : afficher data.data.shape et data.feature_names
    # TODO : renvoyer X, y
    pass

def evaluer_regression(modele, X_train, X_test, y_train, y_test):
    """Entraîne, prédit, renvoie un dict {r2, mae, rmse}.
    Doit renvoyer les 3 métriques de régression vues en section 2.
    """
    # TODO : fit, predict
    # TODO : calculer r2_score, mean_absolute_error, root_mean_squared_error
    # TODO : renvoyer {"r2": ..., "mae": ..., "rmse": ...}
    pass
```

Devrait afficher quelque chose comme :

```
California Housing : (20640, 8) variables, cible = prix médian en centaines de milliers de $
LinearRegression : R2=0.58 MAE=0.53 RMSE=0.75
RandomForest      : R2=0.80 MAE=0.33 RMSE=0.51
```

Checkpoint qualité : testez sur 3 situations.

- Cas normal : le dataset complet, les deux modèles tournent.
- Cas limite : entraînez sur seulement les 100 premières lignes. Le R2 s'effondre-t-il ? Pourquoi un modèle a besoin de données ?
- Cas adversarial : prédisez le prix d'un quartier fictif avec un revenu médian de 0 et 9000 habitants. Le modèle renvoie-t-il une valeur absurde (négative, énorme) ? Que devrait-on faire de ces entrées hors plage en production ?

Phase B : Segmenter les clients d'AirBnB (non supervisé)

Le problème : Airbnb veut regrouper ses annonces en segments naturels (premium, budget, familial...) sans aucune étiquette. C'est du **clustering**, le seul bloc non-supervisé.

Chargez un CSV de listings via son URL, gardez quelques colonnes numériques (prix, nombre de nuits min, nombre d'avis, disponibilité), nettoyez les `NaN` (réflexe J2), **standardisez** (obligatoire avant KMeans, section 6), puis trouvez le bon nombre de segments.

```
import pandas as pd

def charger_airbnb(url_csv):
    """Charge le CSV, garde les colonnes numériques utiles, nettoie les NaN.
    Doit renvoyer un DataFrame propre, sans valeurs manquantes.
    """
    # TODO : pd.read_csv(url_csv)
    # TODO : sélectionner les colonnes numériques pertinentes
    # TODO : gérer les NaN (dropna ou imputation)
    pass

def choisir_k(X_scaled, k_range=range(2, 9)):
    """Pour chaque k, renvoie inertie et silhouette.
    Doit afficher un tableau permettant de repérer le coude / le meilleur k.
    """
    # TODO : pour chaque k, fit KMeans(n_init=10), récupérer inertia_ et silhouette_score
    # TODO : afficher k, inertie, silhouette
    pass
```

Devrait afficher quelque chose comme :

```
Listings chargés : 8000 lignes, 5 colonnes numériques retenues
k=2 : inertie=31200  silhouette=0.41
k=3 : inertie=24800  silhouette=0.46
k=4 : inertie=21500  silhouette=0.39
...
Segment retenu : k=3
```


Ce qui doit casser (et que vous devez gérer) :

- Cas normal : 3 ou 4 segments ressortent, vous décrivez chacun en une phrase ("petits budgets, peu d'avis").
- Cas limite : lancez KMeans **sans standardiser**. Les segments ont-ils encore un sens, ou la colonne "prix" écrase-t-elle tout ? (C'est le piège de la section 6.)
- Cas adversarial : injectez une annonce à 100 000 euros la nuit (une valeur aberrante non nettoyée). Qu'arrive-t-il aux centres de clusters ? Pourquoi le nettoyage de J2 est-il un prérequis absolu ?

Phase C : Courriel vs spam (texte)

Le problème : classer des SMS en spam ou normal. C'est de la **classification de texte**, le terrain de chasse de Naive Bayes (section 7).

Dataset : SMS Spam Collection (UCI). Vectorisez le texte (`CountVectorizer` ou `TfidfVectorizer`), entraînez un Naive Bayes (la baseline texte) et une régression logistique, comparez. Attention : les classes sont **déséquilibrées** (beaucoup plus de messages normaux que de spams), donc l'accuracy seule ment (section 2).

TfidfVectorizer, c'est quoi ? Comme le `CountVectorizer` (bag of words vu en section 7), mais il pondère chaque mot par sa rareté : un mot présent dans tous les messages pèse peu ("le", "de"), un mot rare et discriminant pèse fort ("gratuit", "urgent"). TF = fréquence dans le message, IDF = rareté dans l'ensemble des messages.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

def vectoriser_textes(messages, vectorizer=None):
    """Transforme une liste de textes en matrice numérique.
    Si vectorizer est None, en crée un (fit) ; sinon réutilise (transform).
    Doit renvoyer (X, vectorizer).
    """
    # TODO : créer ou réutiliser le vectorizer
    # TODO : fit_transform ou transform selon le cas
    pass

def evaluer_spam(modele, X_train, X_test, y_train, y_test):
    """Entraîne, prédit, renvoie precision/recall/f1 de la classe spam.
    Doit afficher le classification_report (PAS juste l'accuracy).
    """
    # TODO : fit, predict
    # TODO : afficher classification_report avec precision/recall/f1
    pass
```

Devrait afficher quelque chose comme :

	precision	recall	f1-score	support
normal	0.99	1.00	0.99	966
spam	0.97	0.91	0.94	149
accuracy			0.99	1115

Trois scénarios à passer :

- Happy path : le modèle attrape la grande majorité des spams (recall spam > 0.85).
- Edge case : un message vide `""`. Le vectorizer plante-t-il ? Que renvoie le modèle ?
- Adversarial : un spam déguisé qui imite un message normal (`"salut, ton colis t attend, confirme ici"`). Le modèle se fait-il avoir ? Notez precision ET recall : sur du spam, rater un vrai mail important (faux positif) est souvent pire que laisser passer un spam.

Phase D : Décrypter les signaux d'un sonar (classification binaire)

Le problème : un sous-marin envoie un signal sonar et reçoit un écho. Est-ce une **mine** ou un simple **rocher** ? 60 mesures de fréquence par écho, c'est de la classification binaire sur signal.

Dataset : [Sonar / Connectionist Bench \(Mines vs Rocks\), UCI](#), 208 lignes, 60 variables. Peu de lignes, beaucoup de variables : terrain idéal pour le SVM (section 9), mais aussi un beau cas de test pour tous les algos.

Montez un pipeline qui standardise puis entraîne plusieurs classifieurs (logistique, SVM rbf, Random Forest) et compare.

```
import pandas as pd

def charger_sonar(url):
    """Charge le sonar, sépare X (60 colonnes) et y (M/R -> 1/0).
    Doit renvoyer X, y et afficher la répartition des classes.
    """
    # TODO : pd.read_csv(url, header=None)
    # TODO : dernière colonne = label (M=mine=1, R=rock=0)
    # TODO : renvoyer X, y, afficher la répartition
    pass
```

Devrait afficher quelque chose comme :

```
Sonar : (208, 60), mines=111, rochers=97
LogisticRegression : accuracy=0.76
SVC (rbf)           : accuracy=0.86
RandomForest        : accuracy=0.83
```

Ce qui ne doit pas passer inaperçu :

- Cas normal : le SVM rbf est compétitif (c'est son terrain : peu de données, beaucoup de variables).
- Cas limite : entraînez SANS standardiser. Le SVM et la logistique chutent-ils ? Le Random Forest est-il moins affecté (les arbres se moquent de l'échelle) ?
- Cas adversarial : un écho dont toutes les 60 valeurs sont à zéro (capteur en panne). Le modèle prédit-il quand même une classe avec assurance ? En vrai, un sous-marin se fierait-il à cette prédiction ? Que faudrait-il détecter en amont ?

Phase E : Le Fight des IA (ouverte)

C'est le grand combat, et il n'a pas de fin "terminée" : tant qu'il vous reste du temps, vous allez pouvoir creuser.

La règle du fight : vous choisissez UN des datasets ci-dessus (ou un dataset Kaggle de votre choix), un SEUL découpage train/test (même `random_state` pour tous), et UNE métrique adaptée au problème. Puis vous faites entrer dans l'arène **tous** les algos pertinents et vous sortez un leaderboard trié.

```
def fight_des_ia(X_train, X_test, y_train, y_test, metrique):
    """Entraîne tous les algos sur le MÊME split, renvoie un classement trié.
    metrique : une fonction (y_true, y_pred) -> float.
    Doit afficher un tableau : algo | score | temps d'entraînement (secondes).
    """
    competiteurs = {
        "LogisticRegression": LogisticRegression(max_iter=5000),
        "DecisionTree": DecisionTreeClassifier(random_state=42),
        "RandomForest": RandomForestClassifier(n_estimators=200, random_state=42),
        "GradientBoosting": GradientBoostingClassifier(random_state=42),
        "SVC_rbf": SVC(kernel="rbf"),
        # TODO : ajoutez Naive Bayes si c'est du texte, KMeans si non supervisé
    }
    # TODO : pour chaque algo : fit, predict, chronométrer (time.perf_counter)
    # TODO : calculer metrique(y_test, y_pred)
    # TODO : trier par score décroissant et afficher algo | score | temps
    pass
```

Devrait afficher quelque chose comme :

```
=== LEADERBOARD (jeu : sonar, métrique : F1) ===
1. SVC_rbf : F1=0.87 (0.01s)
2. RandomForest : F1=0.84 (0.31s)
3. GradientBoosting : F1=0.82 (0.42s)
4. LogisticRegression : F1=0.78 (0.02s)
5. DecisionTree : F1=0.71 (0.01s)
```

La métrique agrégée, c'est tout l'enjeu. Votre leaderboard doit afficher, côte à côte : le **score** (la bonne métrique pour CE problème) ET le **temps d'entraînement**. Le champion n'est pas forcément le meilleur score : un modèle 0.2% meilleur mais 40 fois plus lent ne vaut peut-être pas le coup en production. C'est exactement le genre d'arbitrage que vous devrez justifier.

Pour creuser sans fin (choisissez ce qui vous parle) :

- relancez le fight sur un autre dataset de l'après-midi : le podium change-t-il ? Pourquoi ?
- changez la métrique (accuracy vs F1 vs recall) : le champion change-t-il ? Sur le spam, regarder l'accuracy ou le recall donne-t-il le même gagnant ?
- ajoutez une validation croisée (on la verra en détail en J4) pour vérifier que votre podium n'est pas dû à un découpage chanceux
- ajoutez les temps de **prédiction** (pas que d'entraînement) : pour une API en temps réel, c'est ça qui compte
- tunez les hyperparamètres du gagnant (ses réglages internes) avec `GridSearchCV`, l'outil scikit-learn qui teste pour vous toutes les combinaisons de réglages : de combien gagne-t-il ?

💡 **Bonus perso** : sur Kaggle, ce leaderboard que vous venez de construire, c'est exactement ce qui s'affiche en haut d'une compétition, sauf qu'il y a parfois de l'argent au bout. Petit défi hors-cours : ouvrez une compétition "Getting Started" (Titanic ou House Prices) et soumettez le score de votre champion. Vous verrez votre nom sur un vrai leaderboard mondial.

Peer review (relecture par deux)

Avant de committer la version finale, échangez votre notebook avec un binôme. Comm d'habitude ce n'est pas une note, c'est une discussion. La grille porte sur le **raisonnement** et non sur la conformité à une réponse unique (en ML, plusieurs choix sont valides) :

- Le découpage train/test est-il bien le MÊME pour tous les algos du fight ? (Sinon le combat est truqué.)
- La métrique choisie est-elle adaptée au problème ? (Accuracy sur du spam déséquilibré = drapeau rouge.)
- Le champion est-il justifié au-delà du score brut (temps, interprétabilité, robustesse) ?
- Pour le clustering, le binôme a-t-il standardisé avant KMeans ?
- Y a-t-il un choix différent du vôtre (autre algo, autre métrique) qui serait tout aussi défendable ? Lequel, et pourquoi ?

Le but : voir qu'il n'y a pas une seule bonne réponse, et entraîner le réflexe de **justifier** un choix de modèle. C'est exactement ce qu'on vous demandera en soutenance J5.

Récapitulatif Jour 3

Ce qu'on a appris aujourd'hui

1. **Les métriques** : matrice de confusion, precision/recall/F1, ROC/AUC et lift pour la classification ; R^2 /MAE/RMSE pour la régression. Et le piège mortel de l'accuracy sur données déséquilibrées (99% peut être une catastrophe).
2. **Les régressions** : linéaire (la baseline lisible et auditable) et ses limites, polynomiale pour le non-linéaire, Ridge/LASSO pour brider l'overfitting, logistique pour le scoring avec un seuil ajustable.
3. **Le clustering** : KMeans (avec coude et silhouette pour choisir k, et la standardisation obligatoire) et la classification hiérarchique.
4. **Les arbres et compagnie** : arbres de décision lisibles mais fragiles, Naive Bayes imbattable sur le texte.
5. **Les ensembles** : Random Forest (bagging, réduit la variance) vs Gradient Boosting (boosting, réduit le biais). Les champions du tabulaire.
6. **Le SVM** : marge maximale, vecteurs de support, kernel trick pour le non-linéaire. Brille quand on a peu de données et beaucoup de variables.
7. **La règle de décision** : par quel algo commencer selon le problème et la taille des données. Toujours une baseline simple d'abord.
8. **Le Fight des IA** : comparer proprement, même split, même métrique, et justifier son champion au-delà du score brut.

Points d'attention pour demain

- Si votre Arène et votre leaderboard ne sont pas committés sur GitHub, faites-le : c'est la trace de votre travail.
- Demain, un **nouveau combattant entre dans l'arène** : le réseau de neurones. On découvre comment il apprend, et on verra pourquoi, sur nos données tabulaires, il ne bat pas forcément le Gradient Boosting d'aujourd'hui.
- On apprendra aussi à **arbitrer le combat proprement** : train / validation / test, bootstrap et bagging, validation croisée k-fold. Vos podiums d'aujourd'hui sont-ils dus à un découpage chanceux ? Demain on le saura.
- Et on **déploie le champion** : un modèle qui vit sur le web, pas juste dans un notebook.
- Gardez sous le coude votre leaderboard du Fight : il sera le point de départ de l'arbitrage de demain.